Star Pump House Shop Manager

Project overview: what it does, how every part works, and how it is built

Stock · Sales & udhaar · Suppliers · Wholesale shops · Returns · Expenses · Cash closing · Profit & charts · AI assistant · Salesman login

---

Web app · HTML / CSS / JavaScript · Firebase (Auth, Firestore, Hosting, AI Logic, App Check)
Live at star-pump-house.web.app · Document version: October 2026

Contents

1. Introduction
2. How the system is built
3. Modules: what each page does
4. Everyday workflows
5. Data model
6. The AI assistant
7. Security, users and backups
8. Setup, deployment and maintenance
9. Design rules
10. Glossary

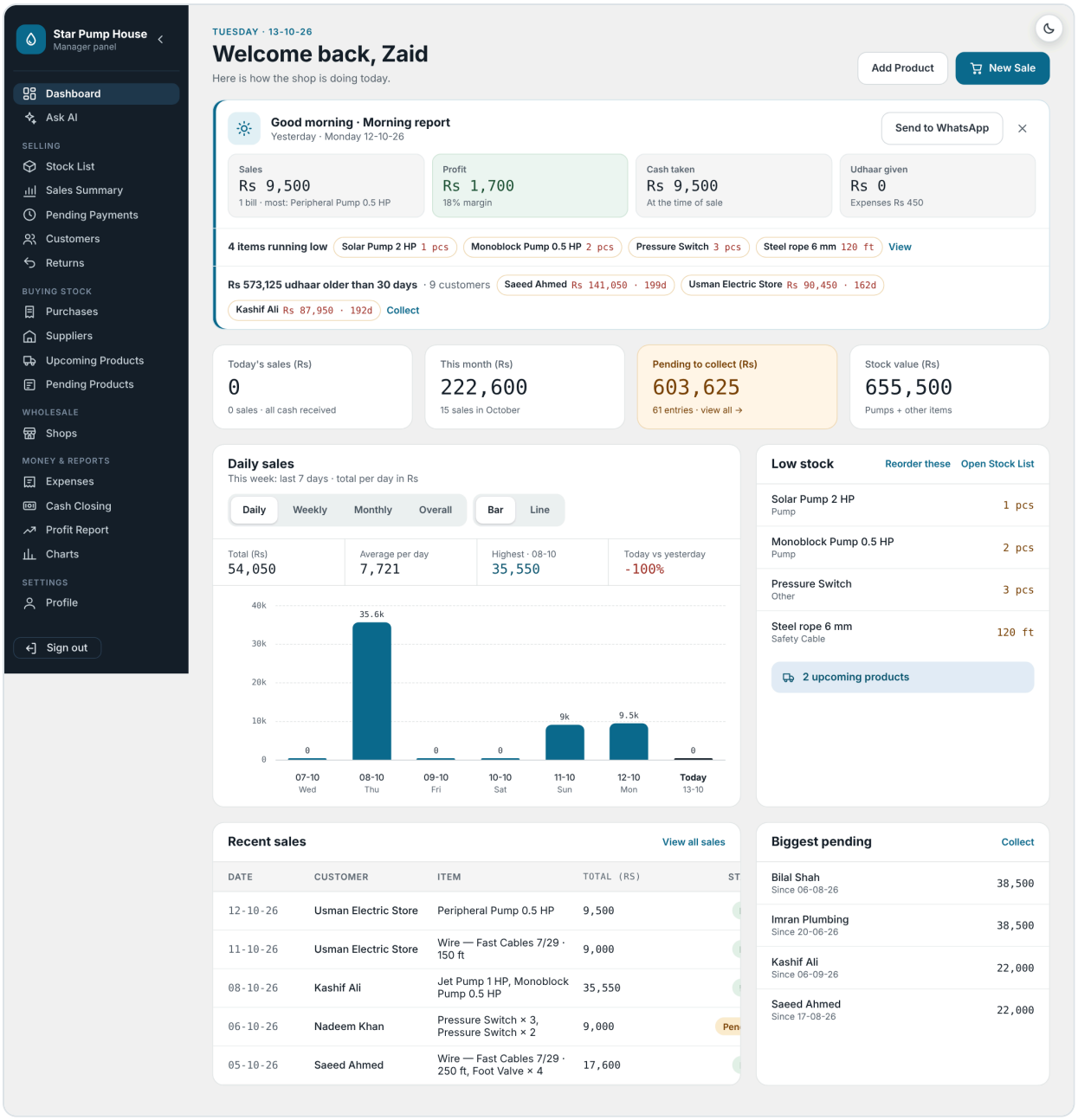
1. Introduction

Star Pump House sells water pumps, wire, safety cable and hardware, and also gives stock on account to other (wholesale) shops. The owner needed a single, free tool to see what is in stock, record every sale, keep track of credit (*udhaar*), suppliers and wholesale accounts, and know the real profit.

This project is that tool: a web app that opens in any browser on a laptop, tablet or phone. It was built step by step by the owner, starting from a simple stock table and growing into a full shop manager.

Main goals

- Free to run (Firebase free plan); data is never lost (cloud database plus JSON backups).
- Simple for daily shop use: big buttons, fast forms, Cash / Loan in one click, works on a phone.
- Every number traceable: stock, cash, udhaar and profit all come from the same records.



Dashboard: morning report, KPIs, sales chart, low stock and pending lists

2. How the system is built

Browser (HTML/CSS/JS pages) → Firebase Auth (login) →

Cloud Firestore (records, cached on the device)

Ask AI page → Firebase AI Logic (Gemini) ⇄ tools that read records in the browser

App Check (reCAPTCHA) → only the real website may use the database and the AI

- **No framework, no build step.** Each page is an HTML file with its own JavaScript module; shared code lives in `common.js`, `utils.js` and helper modules.
- **Live data.** Pages listen to Firestore collections, so a sale made on the phone appears on the laptop at once.
- **Offline-friendly cache.** Firestore keeps a copy of the records on the device, so pages open with data immediately and sync in the background. It is wiped on Sign out.
- **PDFs in the browser.** Receipts, statements and bills are drawn by a tiny built-in PDF writer (`pdf.js`), so no library or server is needed.
- **Hosting.** Firebase Hosting serves the files over HTTPS at a free `*.web.app` address; `firebase deploy` publishes updates.
- **Versions.** Files are loaded with a version number (e.g. `stock.js?v=55`) so browsers always pick up the newest update.

Layer	Technology	Used for
Pages	HTML, CSS, JavaScript (ES modules)	All screens, forms, tables, charts (SVG)
Login	Firebase Authentication	Email/password, verification, password reset, one login per browser
Database	Cloud Firestore	All shop records under users/{uid}/...
Hosting	Firebase Hosting	Free HTTPS website
Assistant	Firebase AI Logic (Gemini)	Questions, answers, filled-in forms
Protection	Firestore rules + App Check	Owner/salesman access, block bots

3. Modules: what each page does

The menu is grouped the way the shop works: **Selling**, **Buying stock**, **Wholesale**, **Money & reports** and **Settings**.

dashboard.html

Dashboard

The first page after login: a quick picture of how the shop is doing.

- **Morning report**: the last working day's sales, profit, cash taken, udhaar given and received, and expenses. It also lists low stock, udhaar older than 30 days, and a reminder if no backup was taken in 7 days. "Send to WhatsApp" sends the report to yourself; "×" hides it for the day.
- **KPIs**: today's sales, this month, pending to collect, and stock value.
- **Sales chart**: Daily / Weekly / Monthly / Overall, as bars or a line, with total, average, highest and change.
- **Low stock**, **Recent sales** and **Biggest pending** lists, each with a shortcut to its page.

stock.html

Stock List

Every product in four categories, with its stock level and price.

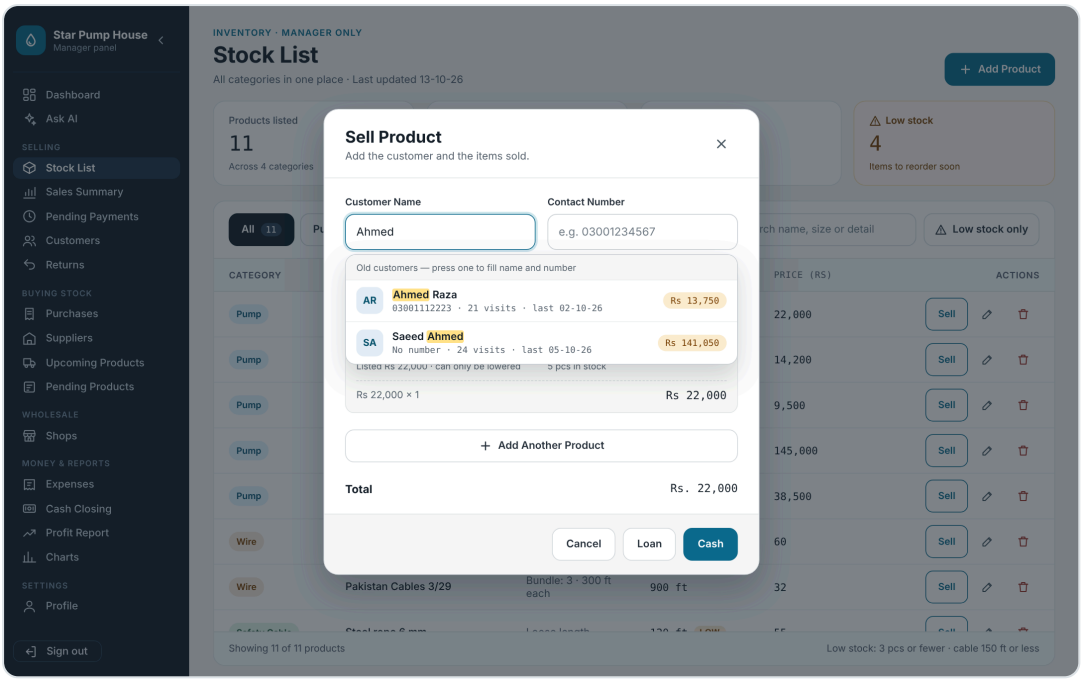
- **Pump**: name, detail, cost, sale price and quantity (pcs).
- **Wire / Safety Cable**: optional name, size, total length in feet and bundles. Feet are split equally between bundles, and the bundle count drops by itself as feet are sold. Prices are per foot.
- **Other**: name and size, then either a quantity (pcs) or a length (ft). Filling one switches the other off.
- A **LOW** badge shows at 3 pcs or fewer, or 150 ft or less. There is search, category tabs and a "Low stock only" filter.
- **Add Product** can also record the purchase from a supplier in the same step. **Edit** and **Delete** ask for confirmation first.

stock.html → Sell

Sell Product

A sale of one or more products to a customer.

- Typing a name or number suggests **old customers**; picking one fills both fields so the sale joins their account.
- **Add Another Product** puts several items on one bill. The price can only be **lowered**, never raised above the listed price.
- Quantity uses − / +; wire and cable ask for feet. Stock goes down when the sale is saved.
- **Cash** asks how much was received. Anything not paid becomes **udhaar** (pending). **Loan** saves everything as udhaar.
- The cost price is saved with the sale, which is how the Profit Report knows the profit later.



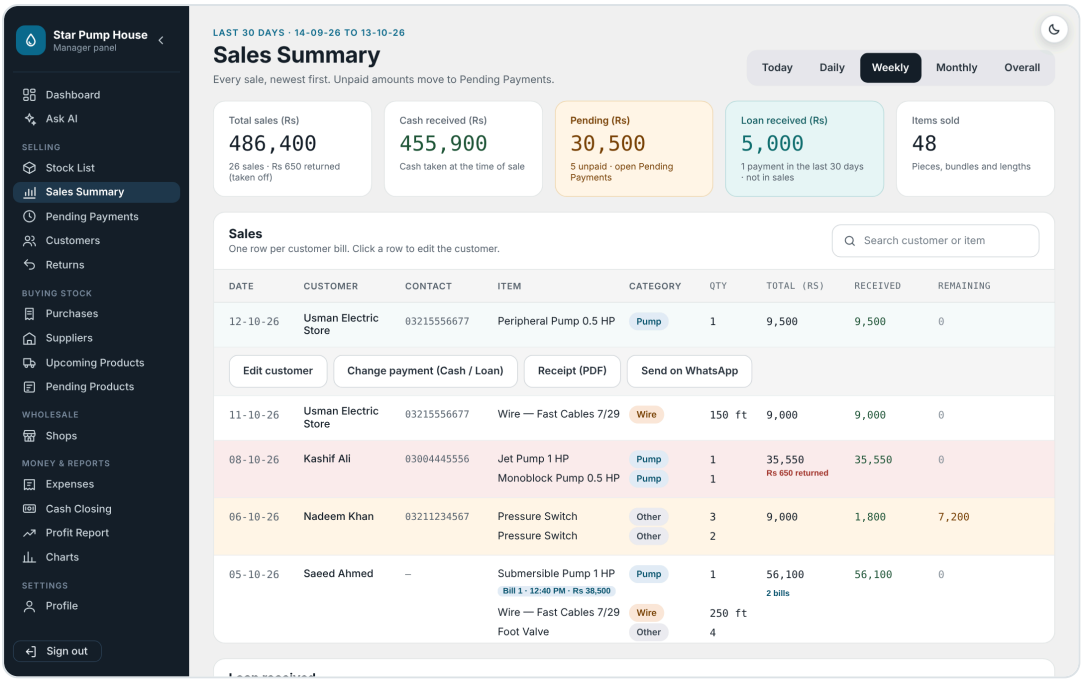
Sell Product: old-customer suggestions and a multi-item bill

sales.html

Sales Summary

Every sale, newest first.

- Periods: **Today · Daily (last 7 days) · Weekly (last 30 days, week by week) · Monthly (12 months) · Overall**.
- KPIs: total sales, cash received at the time of sale, pending, loan received (old udhaar paid back), and items sold.
- Bills of the **same customer on the same day are merged** into one row, with "Bill 1, Bill 2..." inside it. Unpaid rows are amber and returned rows are red.
- Click a row for **Edit customer**, **Change payment** (fix a sale saved as Cash by mistake, or the other way), **Receipt (PDF)**, and **Send on WhatsApp** (PDF file or text message).
- Sales made by a salesman show "by <name>" under the customer.



A sale row opened: edit customer, change payment, receipt and WhatsApp

pending.html

Pending Payments

All udhaar still to be collected.

- Each bill shows the total, paid, remaining and how many days it has been open.
- Receive a full or partial payment. It is logged with the date, so Cash Closing counts it on that day.
- Bulk actions apply to several entries at once.

customers.html

Customers

Customers are created automatically from sales and matched by phone number, or by name when there is no number.

- The list shows total bought, paid, udhaar, visits and last visit, with filters for "All" and "Has udhaar".
- The customer page shows purchases day by day, payments and returns, with buttons for **Receive payment** and **New Sale**.
- There is an **Edit** button for the customer, and a statement PDF (khata).

returns.html

Returns

Goods coming back.

- **From customer**: pick the sale (all returnable sales, newest first, searchable), the quantity and the reason. Then choose **Cash refund**, **Cut udhaar** (if they owe on that sale) or **Exchange**.
- **From shop**: pick the wholesale shop and the item; the value is taken off the shop's bill.
- Untick "Item is fine" for damaged goods: they are kept aside and do not go back into stock.
- The return history and the damaged/kept-aside list are shown on the same page.

purchases.html

Purchases

Bills from suppliers.

- A **New Purchase** has a date, bill number, items (existing products) with cost, and the amount paid now.
- "Add these items to Upcoming Products" sends ordered items to Upcoming. If it is unticked, the items go to Pending Products (planned only).
- Deleting a bill removes it from the supplier's record; stock is not changed.

suppliers.html / supplier.html

Suppliers

Who you buy from and what you owe them.

- The list shows the total bill, paid and remaining for each supplier, with Remaining and Paid-up tabs.
- The supplier page has its bills, payments (cash, bank, cheque, JazzCash/Easypaisa), the products bought from them, and a "Total Bill" PDF.
- Deleting a payment adds it back to what you owe them.

upcoming.html / pending-products.html

Upcoming & Pending Products

Stock between "planned" and "in the shop".

- **Pending Products**: items planned but not ordered yet. Select some and move them to Upcoming when you order them.
- **Upcoming Products**: items ordered and on the way, with the expected date. **Arrived** adds the quantity to the Stock List, or creates the product if it is new.

shops.html / shop.html

Wholesale Shops

Other shops that take stock on account (separate from customer sales).

- A shop has an owner, phone, address and opening balance.
- **Give stock**: items and prices for the shop, with stock going down and an optional amount received now.
- You can receive payments, take shop returns, and see a statement and "Total Bill" PDF.

expenses.html

Expenses

Running costs of the shop.

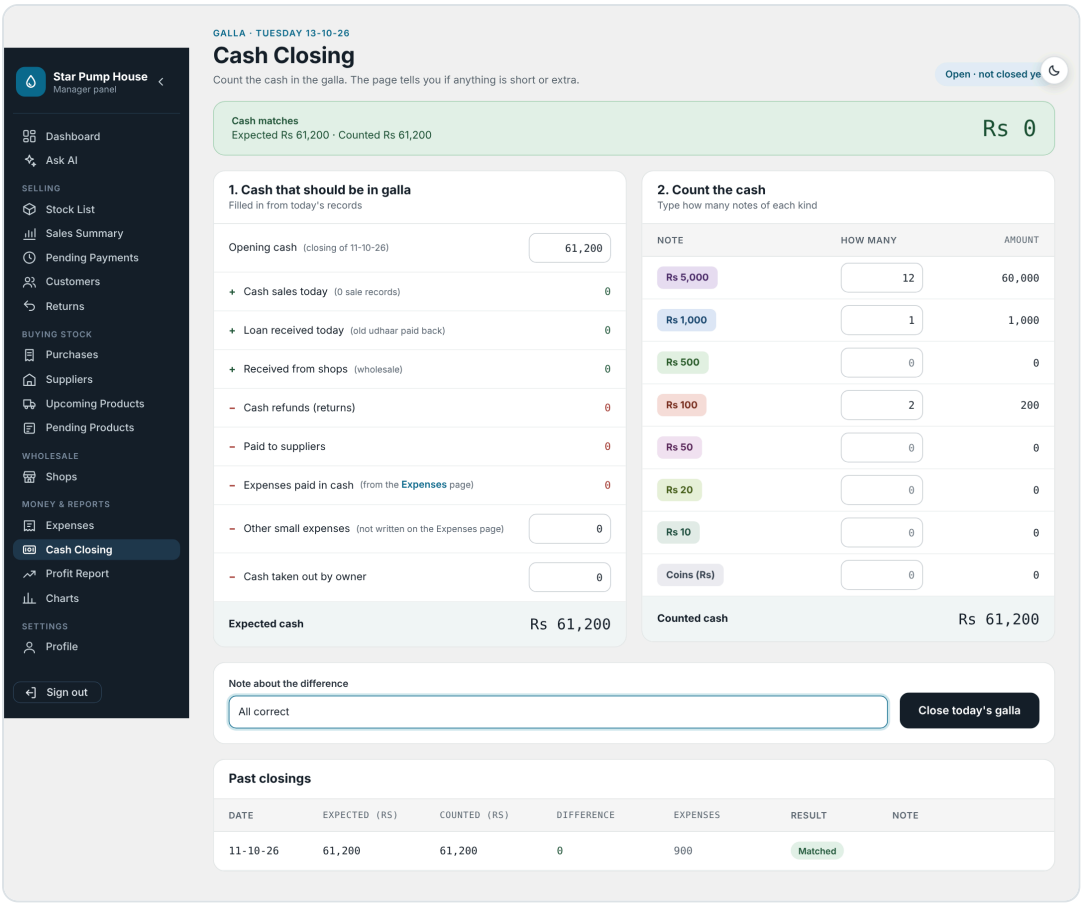
- The kinds are Rent, Electricity / bills, Salary / wages, Tea / food, Transport / fuel, Repairs and Other. Each expense has an amount, date, how it was paid and a note.
- KPIs and a "by kind" bar chart are shown for the chosen period; every expense can be edited or deleted.
- Cash expenses are taken off in Cash Closing, and all expenses are taken off in the net profit.

closing.html

Cash Closing

End-of-day check of the cash drawer (galla).

- **Expected cash** = opening + cash sales today + udhaar received today + received from shops − cash refunds − paid to suppliers − cash expenses − other small expenses − cash taken by the owner.
- Count the notes (5,000 down to 10, plus coins). The page shows whether the cash **matches**, is **short** or is **extra**. Add a note and close the day.
- Tomorrow’s opening cash is taken from today’s counted cash. Past closings are listed.



Cash Closing: expected vs counted cash

profit.html

Profit Report

How much the shop really earned.

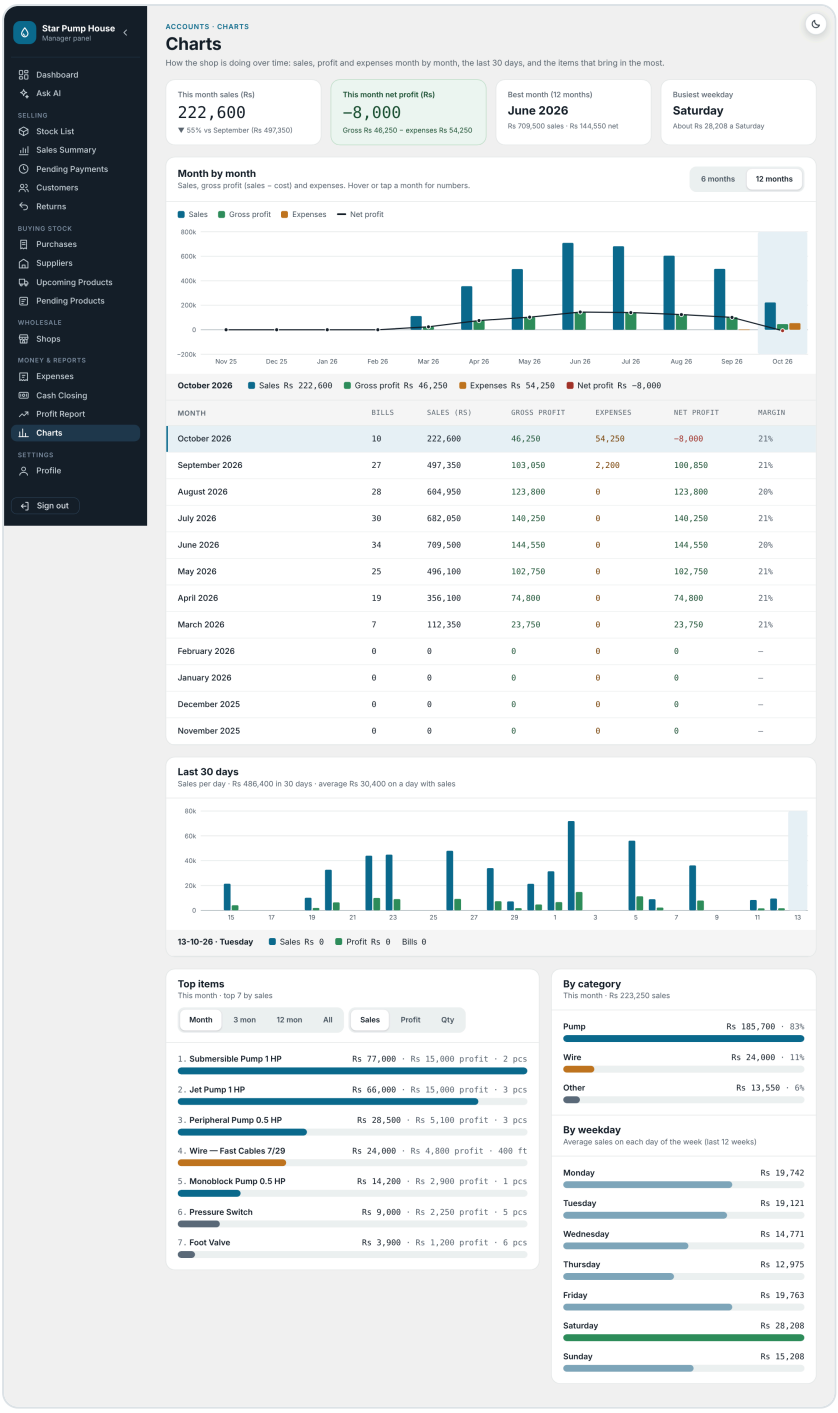
- **Gross profit** = sales − cost of the items sold, with returns taken out.
- **Net profit** = gross profit − expenses (Expenses page + small expenses typed in Cash Closing).
- The report breaks this down by item and category, with the margin %. Sales saved without a cost price are listed separately.

charts.html

Charts

Trends over time.

- **Month by month** (6 or 12 months): sales, gross profit and expenses as bars, with a net profit line. Clicking a month shows its numbers.
- **Last 30 days**: sales and profit per day.
- **Top items** by sales, profit or quantity, for this month, 3 months, 12 months or all time.
- **By category** and **By weekday**: the share of each category, and the average sales on each day of the week.



Charts page

ai.html

Ask AI

The shop's own assistant (see Section 6).

- Ask in English or Roman Urdu, by typing or by voice (mic, UR / EN).
- Answers come from your records. Entries open as forms for you to check and save.
- The chat is kept for 30 minutes; "New chat" starts again.

settings.html

Profile & Settings

Account and shop settings.

- Username, shop name and shop phone (shown in the menu, on receipts and in reports).
- Change password, and email verification (Send email / I have verified).
- **Salesman logins**: add a salesman by name and email, or remove them.
- **Backup**: download a JSON file of all data, or restore from a file. The last backup date is shown here.

4. Everyday workflows

A cash or loan sale

1. Stock List → **Sell** on a product (or Dashboard → New Sale).
2. Type the customer (suggestions appear), add more products if needed, and adjust the price or quantity.
3. Press **Cash** and enter the amount received, or **Loan**.
4. Result: stock goes down. The sale appears in Sales Summary, and any unpaid part goes to Pending Payments and the customer's account. Cash Closing counts the cash taken.

Collecting udhaar

1. Pending Payments or Customers → **Receive payment**.
2. Enter the amount (full or part).
3. Result: the bill's remaining amount goes down. "Loan received" in Sales Summary and Cash Closing include it on that day.

Buying stock

1. Purchases → **New Purchase**: supplier, items, cost and paid now.
2. Ticked "Add to Upcoming": the items wait in Upcoming Products. Unticked: they go to Pending Products.
3. **Arrived** in Upcoming adds the quantity to the Stock List. The supplier's remaining balance updates from the bill and payments.

A customer return

1. Returns → From customer → pick the sale → quantity → reason.
2. Choose Cash refund, Cut udhaar or Exchange; untick for damaged items.
3. Result: stock comes back (unless damaged), and the money is refunded or cut from udhaar. Sales and profit reports take the returned value off.

Wholesale shop on account

1. Shops → open the shop → **Give stock** (items, prices, cash received now).
2. Later: **Receive payment**, or a shop return from the Returns page.
3. Result: stock goes down, and the shop's khata shows the bills, payments and remaining balance.

Closing the day

1. Cash Closing at night: check the expected cash lines.
2. Count the notes; read Matches / Short / Extra, write a note and press **Close today's galla**.
3. The counted cash becomes tomorrow's opening cash.

Adding a salesman

1. Profile → Salesman logins → name + email → **Add salesman** (the new Firestore rules must be published first).
2. The salesman signs up with that email and logs in.
3. They see only the Stock List and can only sell; their sales show "by <name>". **Remove** ends their access.

5. Data model

Each owner's records live under `users/{uid}/` in Firestore. A sale with several products is saved as one document per product line; lines of the same bill share a `timestamp`.

Collection	Main fields	Used by
products	category, name, brand/size, quantity or length (ft), bundle, perBundle, price, cost	Stock, Sell, reports
sales	productName, quantity / lengthFt, amount, received, remainingCost, paidAtSale, status, customerName, customerContact, cost, soldBy, date, time, timestamp	Sales, Pending, Customers, Profit, Charts, Closing
payments	amount, date, billKey / saleIds	Udhaar received
returns	source (customer/shop), type (refund/udhaar/exchange/cut), qty, value, refund, udhaarCut, backToStock	Returns, reports
suppliers · purchases · supplierPayments	name, phone, openingBalance · items, total, paid, billNo · amount, method	Purchases, Suppliers
upcoming · pendingProducts	item, qty, cost, supplier, expected date	Upcoming, Pending Products
shops · shopBills · shopPayments	name, owner, openingBalance · items, total, received · amount, method	Wholesale shops
expenses	date, kind, amount, method, note	Expenses, Profit, Closing
closings	date (doc id), opening, expected, counted, diff, expenses	Cash Closing
settings/shop	shopName, shopPhone, lastBackup	Menu, receipts, reminders
staff (+ top-level staffIndex)	name, email, ownerUid	Salesman logins

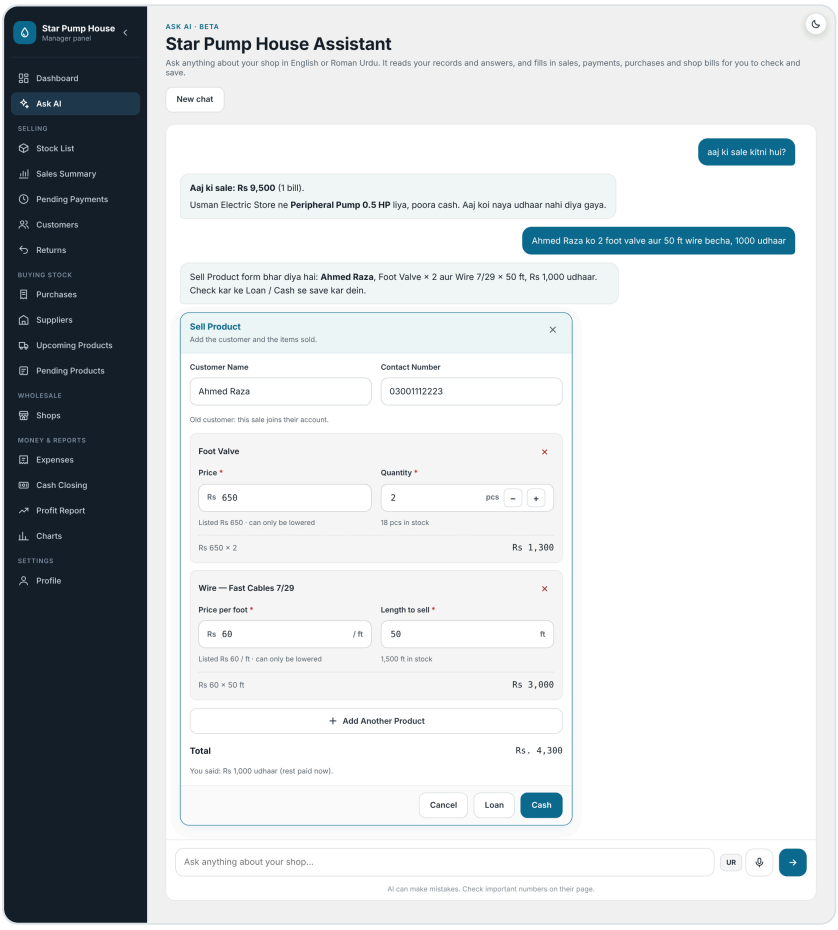
How the key numbers are worked out

- **Remaining (udhaar)** of a sale = amount − paid at sale − payments received later − udhaar cut by returns.
- **Cash received** (Sales Summary) = cash taken at the moment of sale; udhaar paid back later is shown separately as **Loan received**.
- **Stock value** = price × quantity for pumps and other items; wire/cable shown as feet in stock.
- **Gross profit** = sale amount − saved cost (returns taken out); **Net profit** = gross profit − expenses.
- **Supplier remaining** = opening balance + bills − payments; **Shop remaining** = opening balance + stock given − payments − returns.

6. The AI assistant

The **Ask AI** page is the shop's own assistant, named after the shop (for example "Star Pump House Assistant"). It uses **Firebase AI Logic** with Google Gemini models and **function calling**.

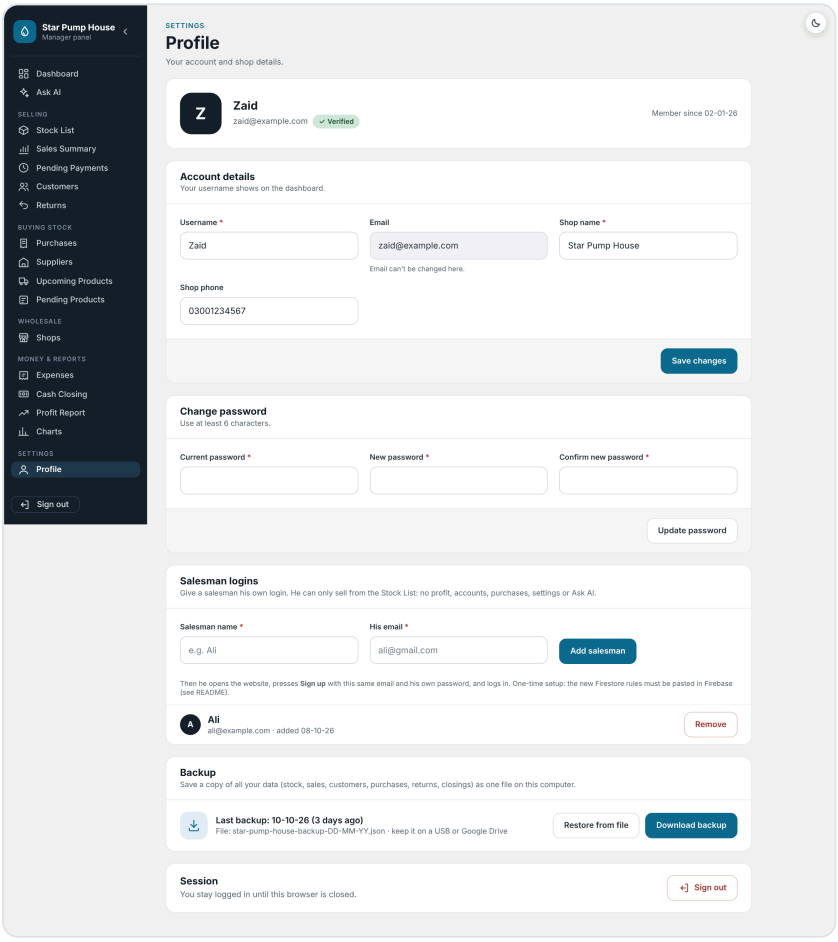
- **It reads, it does not guess.** The model never receives the whole database. It calls small "tools" that run in the browser over your records: shop overview, find sales, customer account, udhaar list, stock lookup, best sellers, sales by period, profit, expenses, suppliers, shops, purchases, returns, orders, daily cash, and stock and sales forecasts.
- **It never saves on its own.** For entries it opens the **same form the normal page uses**, already filled in. The owner checks it and presses Save. Forms available: sale, udhaar received, purchase, supplier/shop payment, give stock to a shop, customer return, shop return, change payment, edit customer, add/edit expense, add/edit product, add supplier/shop, and delete a record (with confirmation).
- **Choices as buttons.** If a name matches several customers, products or sales, it shows buttons; one tap opens the right form.
- **Language.** It answers in the language of the question (English or Roman Urdu), and you can speak instead of typing (Urdu or English).
- **Free-tier friendly.** It switches between several Gemini models when one is busy, and waits and retries on rate limits. The chat is kept for 30 minutes.



Ask AI: an answer from the records, then a filled-in Sell form for the owner to check and save

7. Security, users and backups

- **Login on every page.** Pages stay hidden (a small loader shows) until the login is confirmed. The session ends when the browser closes, and logging in on a new tab signs the other tabs of that browser out. Sign out asks for confirmation and wipes the records cached on the device.
- **Owner vs salesman.** The owner adds a salesman by email (Profile). The salesman signs up with that email and can then only open the Stock List and sell. The Firestore rules enforce this too: a salesman may read products, sales and the shop name, create sales, and only lower stock quantities.
- **Firestore rules.** Every owner can only reach `users/{their uid}/…`.
- **App Check.** A reCAPTCHA Enterprise key proves requests come from the real website. Once the metrics look clean it is enforced for Firestore and AI Logic, which blocks bots and protects the free AI quota.
- **Backups.** Profile → **Download backup** saves every collection as one JSON file; **Restore** writes it back. The Dashboard reminds the owner if no backup was taken in 7 days.
- **New owners** are asked for their shop's name on first login; it appears in the menu, on receipts and in reports.



Profile: account, password, salesman logins and backup

8. Setup, deployment and maintenance

1. Create a Firebase project (free Spark plan). Enable **Email/Password** sign-in.
2. Create the **Firestore** database and publish `firestore.rules`.
3. Register a **Web app** and paste its config into `firebase-config.js`.
4. Enable **AI Logic** (Gemini Developer API) for the assistant.
5. Install the Firebase CLI, run `firebase init hosting` (public folder = the project files), then `firebase deploy`.
6. Optional: App Check (reCAPTCHA Enterprise key → Firebase App Check → monitor → enforce).
7. Optional: a nicer free address (an extra Hosting site such as `starpumphouse.web.app`) or a paid custom domain. Add any new domain to Authentication → Authorized domains and to the reCAPTCHA key.

Updating the site: replace the changed files → `firebase deploy` → hard refresh (Cmd/Ctrl + Shift + R). Because files carry version numbers, phones pick up the update after a normal reload.

Free-plan limits (approx.): Firestore 50,000 reads and 20,000 writes per day; Hosting 10 GB storage and 360 MB/day transfer. The device cache keeps reads low; a single shop stays well inside these limits.

9. Design rules

- Colours: primary blue #0E6A8C, dark menu #14202B, light background #F3F4F1. Amber means pending/udhaar, green means received, red means returns/short. Dark mode is available on every page.
- Fonts: IBM Plex Sans for text and IBM Plex Mono for numbers. Rounded cards and a collapsible sidebar; on phones a slide-in menu and an optional "Laptop view".
- Numbers have commas and money fields accept digits only. Dates are DD-MM-YY. Required fields have a red ✱. Text starts with a capital letter. Lists are newest first and search highlights matches. Interface text is in English.

10. Glossary

Word	Meaning
Udhaar	Credit: money a customer still owes for goods taken
Galla	The shop’s cash drawer
Khata	Account statement of a customer, supplier or shop
Loan (sale)	A sale saved as udhaar with nothing (or part) paid
Wholesale shop	Another shop that takes stock on account
Upcoming	Stock ordered and on the way
Pending Products	Stock planned but not ordered
Cash Closing	End-of-day count of the galla against the expected cash

Screenshots in this document use demo data.